

Travaux Pratiques – Parcourir le système de fichiers Linux et les paramètres d'autorisation

Objectifs

Au cours de ces travaux pratiques, vous allez vous familiariser avec les systèmes de fichiers Linux.

Ressources requises

- Poste de travail virtuel CyberOps

Partie 1 : Explorer les systèmes de fichiers sous Linux

Le système de fichiers Linux est l'une des fonctionnalités les plus populaires du système d'exploitation. Linux prend en charge différents types de systèmes de fichiers, mais ce TP se concentre sur **ext**, l'un des types de systèmes de fichiers les plus communément utilisés sous Linux.

Étape 1 : Accédez à la ligne de commande.

Lancez le poste de travail virtuel CyberOps et ouvrez une fenêtre du terminal.

Étape 2 : Affichez les systèmes de fichiers actuellement montés.

Les systèmes de fichiers doivent être *montés* pour que vous puissiez y accéder et les utiliser. En informatique, *monter un système de fichiers* signifie le rendre accessible au système d'exploitation. Le montage d'un système de fichiers consiste à associer la partition physique du périphérique de type bloc (disque dur, disque SSD, clé USB, etc.) à un répertoire à partir duquel le système de fichiers complet peut être consulté. Étant donné que le répertoire mentionné précédemment devient la racine du système de fichiers que vous venez de monter, il est également connu sous le nom de *point de montage*.

- a. Utilisez la commande **lsblk** pour afficher tous les périphériques de type bloc :

```
[analyst@secOps ~]$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINT
sda 8:0 0 5.9G 0 disk
└─sda1 8:1 0 5.9G 0 part /
sdb 8:16 0 1G 0 disk
└─sdb1 8:17 0 1023M 0 part
sr0 11:0 1 1024M 0 rom
```

La sortie ci-dessus montre que trois périphériques de type bloc sont installés sur le poste de travail virtuel CyberOps : sr0, sda et sdb. La sortie en arborescence affiche également des partitions sous sda et sdb. De manière conventionnelle, /dev/sdX est utilisé par Linux pour représenter les disques durs. Le numéro à la fin indique le numéro de partition au sein du périphérique. Les ordinateurs disposant de plusieurs disques durs afficheraient logiquement davantage de périphériques /dev/sdX. Si Linux était en cours d'exécution sur un ordinateur équipé de quatre disques durs par exemple, ces disques durs s'afficheraient par défaut en tant que /dev/sda, /dev/sdb, /dev/sdc et /dev/sdd. La sortie implique que sda et sdb sont des disques durs, contenant chacun une seule partition. Le résultat indique également que sda est un disque de 5,9 Go, tandis que sdb dispose de 1 Go.

Remarque : Linux affiche également souvent des clés USB en tant que /dev/sdX, selon leur type de micrologiciel.

- b. Utilisez la commande **mount** pour afficher des informations plus détaillées sur les systèmes de fichiers actuellement montés sur le poste de travail virtuel CyberOps.

```
[analyst@secOps ~]$ mount
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
sys on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
dev on /dev type devtmpfs (rw,nosuid,relatime,size=1030408k,nr_inodes=218258,mode=755)
run on /run type tmpfs (rw,nosuid,nodev,relatime,mode=755)
/dev/sda1 on / type ext4 (rw,relatime,data=ordered)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000)
tmpfs on /sys/fs/cgroup type tmpfs (ro,nosuid,nodev,noexec,mode=755)
<résultat omis>
```

Bon nombre des systèmes de fichiers ci-dessus ne sont pas abordés dans ce cours. Ils ne sont donc d'aucune utilité pour ce TP. Concentrons-nous sur le *système de fichiers racine*, celui qui est stocké dans **/dev/sda1**. Le système de fichiers racine contient le système d'exploitation Linux lui-même. Tous les programmes, outils et fichiers de configuration sont stockés par défaut dans le système de fichiers racine.

- c. Exécutez à nouveau la commande **mount**, mais cette fois, utilisez le symbole de barre verticale **|** pour envoyer les résultats de la commande vers **grep** afin de filtrer les résultats et d'afficher uniquement le système de fichiers racine :

```
[analyst@secOps ~]$ mount | grep sda1
/dev/sda1 on / type ext4 (rw,relatime,data=ordered)
```

Dans les résultats filtrés ci-dessus, la commande **mount** indique que le système de fichiers racine se trouve dans la première partition du périphérique de type bloc sda (**/dev/sda1**). Nous savons qu'il s'agit du système de fichiers racine en raison du point de montage utilisé : « **/** » (le symbole de barre oblique). Les résultats nous révèlent aussi le type de mise en forme utilisée dans la partition, **ext4** dans ce cas. Les informations entre parenthèses font référence aux options de montage de la partition.

- d. Exécutez les deux commandes ci-dessous depuis le **poste de travail virtuel CyberOps** :

```
[analyst@secOps ~]$ cd /
[analyst@secOps /]$ ls -l
```

Que nous révèlent les résultats ? À quel endroit les fichiers répertoriés sont-ils stockés physiquement ?

Pourquoi **/dev/sdb1** ne s'affiche-t-il pas dans les résultats ci-dessus ?

Étape 3 : Montez et démontez manuellement des systèmes de fichiers.

La commande **mount** peut également être utilisée pour monter et démonter des systèmes de fichiers. Comme nous l'avons vu à l'étape 1, deux disques durs sont installés sur le poste de travail virtuel CyberOps. Le premier a été identifié par le noyau comme **/dev/sda** et le second comme **/dev/sdb**. Pour pouvoir être monté, un périphérique de type bloc doit disposer d'un point de montage.

- a. Utilisez la commande **ls -l** pour vérifier que le répertoire **second_drive** se trouve dans le répertoire de base de l'analyste.

```
[analyst@secOps ~]$ cd ~
[analyst@secOps ~]$ ls -l
total 28
drwxr-xr-x 3 analyst analyst 4096 Aug 16 15:15 cyops_folder2
drwxr-xr-x 2 analyst analyst 4096 Sep 26 2014 Desktop
drwx----- 3 analyst analyst 4096 Jul 14 11:28 Downloads
drwxr-xr-x 8 analyst analyst 4096 Jul 25 16:27 lab.support.files
drwxr-xr-x 2 analyst analyst 4096 Mar 3 15:56 second_drive
-rw-r--r-- 1 analyst analyst 142 Aug 16 15:11 some_text_file.txt
-rw-r--r-- 1 analyst analyst 254 Aug 16 13:38 space.txt
```

Remarque : si le répertoire **second_drive** n'existe pas, utilisez la commande **mkdir second_drive** pour le créer.

```
[analyst@secOps ~]$ mkdir second_drive
```

Remarque : selon l'état de votre machine virtuelle, d'autres fichiers et répertoires peuvent s'afficher.

- b. Utilisez à nouveau la commande **ls -l** pour répertorier le contenu du répertoire **second_drive** que vous venez de créer.

```
[analyst@secOps ~]$ ls -l second_drive/
total 0
```

Vous remarquerez que le répertoire est vide.

- c. Utilisez la commande **mount** pour monter **/dev/sdb1** sur le répertoire **second_drive** que vous venez de créer. La syntaxe de montage est la suivante : **mount [options] <appareil à monter> <point de montage>**.

```
[analyst@secOps ~]$ sudo mount /dev/sdb1 ~/second_drive/
[sudo] password for analyst:
```

Aucun résultat ne s'affiche, ce qui signifie que le processus de montage s'est exécuté avec succès.

- d. Maintenant que **/dev/sdb1** a été monté sur **/home/analyst/second_drive**, utilisez la commande **ls -l** pour répertorier à nouveau le contenu du répertoire.

```
[analyst@secOps ~]$ ls -l second_drive/
total 20
drwx----- 2 root root 16384 Mar 3 10:59 lost+found
-rw-r--r-- 1 root root 183 Mar 3 15:42 myFile.txt
```

Pourquoi le répertoire n'est-il plus vide ? À quel endroit les fichiers répertoriés sont-ils stockés physiquement ?

- e. Exécutez la commande **mount** à nouveau sans aucune option pour afficher des informations détaillées sur la partition **/dev/sdb1**. Comme vous l'avez fait précédemment, utilisez la commande **grep** pour afficher uniquement les systèmes de fichiers **/dev/sdX** :

```
[analyst@secOps ~]$ mount | grep sd
/dev/sda1 on / type ext4 (rw,relatime,data=ordered)
/dev/sdb1 on /home/analyst/second_drive type ext2
(rw,relatime,block_validity,barrier,user_xattr,acl)
```

- f. Le démontage des systèmes de fichiers est tout aussi simple. Assurez-vous de remplacer le répertoire par un élément situé en dehors du point de montage et utilisez la commande **umount** comme indiqué ci-dessous :

```
[analyst@secOps ~]$ sudo umount /dev/sdb1
[sudo] password for analyst:
[analyst@secOps ~]$
[analyst@secOps ~]$ ls -l second_drive/
total 0
```

Partie 2 : Autorisations de fichiers

Les systèmes de fichiers Linux possèdent des fonctions intégrées qui définissent la capacité des utilisateurs à afficher, à modifier, à explorer et à exécuter le contenu du système de fichiers. En clair, chaque fichier contenu dans les systèmes de fichiers contient son propre ensemble d'autorisations, c'est-à-dire un ensemble de définitions qui indiquent de quelle manière les utilisateurs et les groupes peuvent interagir avec le fichier.

Étape 1 : Afficher et modifier les autorisations des fichiers.

- a. Accédez à /home/analyst/lab.support.files/scripts/.

```
[analyst@secOps ~]$ cd lab.support.files/scripts/
```

- b. Utilisez la commande **ls -l** pour afficher les autorisations des fichiers.

```
[analyst@secOps scripts]$ ls -l
total 60
-rwxr-xr-x 1 analyst analyst 190 Jun 13 09:45 configure_as_dhcp.sh
-rwxr-xr-x 1 analyst analyst 192 Jun 13 09:45 configure_as_static.sh
-rwxr-xr-x 1 analyst analyst 3459 Jul 18 10:09 cyberops_extended_topo_no_fw.py
-rwxr-xr-x 1 analyst analyst 4062 Jul 18 10:09 cyberops_extended_topo.py
-rwxr-xr-x 1 analyst analyst 3669 Jul 18 10:10 cyberops_topo.py
-rw-r--r-- 1 analyst analyst 2871 Apr 28 11:27 cyops.mn
-rwxr-xr-x 1 analyst analyst 458 May 1 13:50 fw_rules
-rwxr-xr-x 1 analyst analyst 70 Apr 28 11:27 mal_server_start.sh
drwxr-xr-x 2 analyst analyst 4096 Jun 13 09:55 net_configuration_files
-rwxr-xr-x 1 analyst analyst 65 Apr 28 11:27 reg_server_start.sh
-rwxr-xr-x 1 analyst analyst 189 Dec 15 2016 start_ELK.sh
-rwxr-xr-x 1 analyst analyst 85 Dec 22 2016 start_miniedit.sh
-rwxr-xr-x 1 analyst analyst 76 Jun 22 11:38 start_pox.sh
-rwxr-xr-x 1 analyst analyst 106 Jun 27 09:47 start_snort.sh
-rwxr-xr-x 1 analyst analyst 61 May 4 11:45 start_tftpd.sh
```

Prenez le fichier **cyops.mn** comme exemple. Qui est le propriétaire du fichier ? Qu'en est-il du groupe ?

Les autorisations pour **cyops.mn** sont **-rw-r--r--**. Qu'est-ce que cela signifie ?

- c. La commande **touch** est extrêmement simple et utile. Elle permet de créer rapidement un fichier texte vide. Utilisez la commande suivante pour créer un fichier vide dans le répertoire **/mnt** :

```
[analyst@secOps scripts]$ touch /mnt/myNewFile.txt
touch: cannot touch '/mnt/myNewFile.txt': Autorisation refusée
```

Le fichier a-t-il été créé ? Répertoriez les autorisations, les paramètres de propriété et le contenu du répertoire **/mnt** et expliquez ce qu'il s'est passé. Notez les réponses sur les lignes ci-dessous.

```
[analyst@secOps ~]$ ls -l /mnt
total 4
drwxr-xr-x 2 root root 4096 Mar  3 11:13 second_drive
```

Si nous ajoutons l'option **-d**, la commande répertorie les autorisations du répertoire parent.

```
[analyst@secOps ~]$ ls -ld /mnt
drwxr-xr-x 3 root root 4096 Mar  3 15:43 /mnt
```

Que peut-on faire pour que la commande **touch** ci-dessus aboutisse ?

- d. La commande **chmod** permet de modifier les autorisations d'un fichier ou d'un répertoire. Comme précédemment, montez la partition **/dev/sdb1** sur le répertoire **/home/analyst/second_drive** créé précédemment dans ce TP :

```
[analyst@secOps scripts]$ sudo mount /dev/sdb1 ~/second_drive/
```

- e. Remplacez le répertoire par **second_drive** et répertoriez le contenu de celui-ci :

```
[analyst@secOps scripts]$ cd ~/second_drive
[analyst@secOps second_drive]$ ls -l
total 20
drwx----- 2 root root 16384 Mar  3 10:59 lost+found
-rw-r--r-- 1 root root 183 Mar  3 15:42 myFile.txt
```

Quelles sont les autorisations du fichier **myFile.txt** ?

- f. Utilisez la commande **chmod** pour modifier les autorisations du fichier **myFile.txt**.

```
[analyst@secOps second_drive]$ sudo chmod 665 myFile.txt
[analyst@secOps second_drive]$ ls -l
total 20
drwx----- 2 root root 16384 Mar  3 10:59 lost+found
-rw-rw-r-x 1 root root 183 Mar  3 15:42 myFile.txt
```

Les autorisations ont-elles été modifiées ? Quelles sont les autorisations du fichier **myFile.txt** ?

La commande **chmod** exprime les autorisations par chaînes de huit caractères. Ainsi, 665 se décompose de la façon suivante :

6 en chaîne de huit caractères équivaut à 110 en chaîne binaire. En supposant que chaque position des autorisations d'un fichier peut être 1 ou 0, 110 signifie rw- (lire=1, écrire=1 et exécuter=0).

Par conséquent, la commande **chmod 665 myFile.txt** modifie les autorisations tel que suit :

Propriétaire : rw- (6 ou 110 en chaîne de huit caractères)

Groupe : rw- (6 ou 110 en chaîne de huit caractères)

Autre : r-x (5 ou 101 en chaîne de huit caractères)

Quelle commande modifierait les autorisations du fichier myFile.txt sur rwxrwxrwx pour fournir à tous les utilisateurs du système un accès complet au fichier ?

- g. La commande **chown** permet de modifier la propriété d'un fichier ou d'un répertoire. Saisissez la commande ci-dessous pour définir l'analyste comme propriétaire du fichier **myFile.txt** :

```
[analyst@secOps second_drive]$ sudo chown analyst myFile.txt
[sudo] password for analyst:
[analyst@secOps second_drive]$ ls -l
total 20
drwx----- 2 root root 16384 Mar 3 10:59 lost+found
-rw-rw-r-x 1 analyst root 183 Mar 3 15:42 myFile.txt
[analyst@secOps second_drive]$
```

Remarque : pour configurer en même temps l'analyste comme propriétaire et comme groupe, utilisez le format **sudo chown analyst:analyst myFile.txt**.

- h. Maintenant que l'analyste est le propriétaire du fichier, essayez d'ajouter le mot « test » à la fin de **myFile.txt**.

```
[analyst@secOps second_drive]$ echo test >> myFile.txt
[analyst@secOps second_drive]$ cat myFile.txt
```

L'opération a-t-elle réussi ? Expliquez votre réponse.

Étape 2 : Répertoire et autorisations

À l'instar des fichiers ordinaires, les répertoires possèdent également des autorisations. Leurs autorisations contiennent cependant un bit supplémentaire.

- a. Redéfinissez le répertoire sur /home/analyst/lab.support.files et exécutez la commande **ls -l** pour répertorier tous les fichiers de manière détaillée :

```
[analyst@secOps second_drive]$ cd ~/lab.support.files/
[analyst@secOps lab.support.files]$ ls -l
total 580
-rw-r--r-- 1 analyst analyst 649 Jun 28 18:34 apache_in_epoch.log
-rw-r--r-- 1 analyst analyst 126 Jun 28 11:13 applicationX_in_epoch.log
drwxr-xr-x 4 analyst analyst 4096 Aug 7 15:29 attack_scripts
-rw-r--r-- 1 analyst analyst 102 Jul 20 09:37 confidential.txt
-rw-r--r-- 1 analyst analyst 2871 Dec 15 2016 cyops.mn
-rw-r--r-- 1 analyst analyst 75 May 24 11:07 elk_services
-rw-r--r-- 1 analyst analyst 373 Feb 16 16:04 h2_dropbear.banner
-rw-r--r-- 1 analyst analyst 147 Mar 21 15:30 index.html
-rw-r--r-- 1 analyst analyst 255 May 2 13:11 letter_to_grandma.txt
```

```
-rw-r--r-- 1 analyst analyst 24464 Feb  7  2017 logstash-tutorial.log
drwxr-xr-x 2 analyst analyst  4096 May 25 13:01 malware
-rwxr-xr-x 1 analyst analyst   172 Jul 25 16:27 mininet_services
drwxr-xr-x 2 analyst analyst  4096 Feb 14  2017 openssl_lab
drwxr-xr-x 2 analyst analyst  4096 Aug  7 15:25 pcaps
drwxr-xr-x 7 analyst analyst  4096 Sep 20  2016 pox
-rw-r--r-- 1 analyst analyst 473363 Feb 16 15:32 sample.img
-rw-r--r-- 1 analyst analyst    65 Feb 16 15:45 sample.img_SHA256.sig
drwxr-xr-x 3 analyst analyst  4096 Jul 18 10:10 scripts
-rw-r--r-- 1 analyst analyst 25553 Feb 13  2017 SQL_Lab.pcap
```

Comparez les autorisations du répertoire **malware** avec le fichier **mininet_services**. En quoi leurs autorisations diffèrent-elles ?

La lettre « d » indique que cette entrée spécifique est un répertoire et non un fichier. Le bit d'exécution est également différent entre les autorisations du fichier et celles du répertoire. Si le bit d'exécution d'un fichier est activé, cela signifie que le fichier peut être exécuté par le système. Les répertoires ont un comportement différent des fichiers lorsque leur bit d'exécution est défini (un fichier avec le bit d'exécution défini est un programme ou un script exécutable). Un répertoire dont le bit d'exécution est défini indique si un utilisateur peut accéder à ce répertoire.

Les commandes **chmod** et **chown** s'appliquent aux répertoires de la même manière qu'elles s'appliquent aux fichiers.

Partie 3 : Liens symboliques et autres types de fichiers spéciaux

Nous vous avons présenté plusieurs types de fichiers Linux. Dans une commande **ls -l**, le premier caractère de chaque liste de fichiers indique le type de fichier. Voici les trois types de fichiers Linux, avec leurs sous-types et leurs caractères :

- **Les fichiers ordinaires (-)**, y compris :
 - Les fichiers lisibles - fichiers texte
 - Les fichiers binaires - programmes
 - Les fichiers image
 - Les fichiers compressés
- **Les fichiers de répertoire (d)**
 - Les dossiers
- **Les fichiers spéciaux**, y compris :
 - **Les fichiers de périphérique de type bloc (b)** : utilisés pour accéder au matériel physique, par exemple des points de montage pour accéder aux disques durs.
 - **Les fichiers de périphérique de type caractère (c)** : fichiers qui fournissent un flux série d'entrée et de sortie, par exemple les terminaux tty.
 - **Les fichiers de périphérique de type pipe (p)** : fichiers utilisés pour transmettre des informations dans l'ordre de réception des octets. On appelle également cela la méthode FIFO (pour First In, First Out, ou Premier entré, premier sorti).
 - **Les fichiers de lien symbolique (l)** : fichiers utilisés pour établir une liaison avec d'autres fichiers ou répertoires. Il en existe deux types : les liens symboliques et les liens matériels.

- **Les fichiers de socket** : fichiers utilisés pour transmettre des informations entre les différentes applications souhaitant communiquer sur le réseau.

Étape 1 : Examinez les types de fichiers.

- a. Utilisez la commande **ls -l** pour afficher les fichiers. Vous remarquerez que le premier caractère de chaque ligne est soit un « - » (qui indique un fichier), soit un « d » (qui indique un répertoire).

```
[analyst@secOps ~]$ ls -l
total 28
drwxr-xr-x 3 analyst analyst 4096 Aug 16 15:15 cyops_folder2
drwxr-xr-x 2 analyst analyst 4096 Sep 26 2014 Desktop
drwx----- 3 analyst analyst 4096 Jul 14 11:28 Downloads
drwxr-xr-x 8 analyst analyst 4096 Jul 25 16:27 lab.support.files
drwxr-xr-x 3 analyst analyst 4096 Mar 3 18:23 second_drive
-rw-r--r-- 1 analyst analyst 142 Aug 16 15:11 some_text_file.txt
-rw-r--r-- 1 analyst analyst 254 Aug 16 13:38 space.txt
```

- b. Récupérez un listing du répertoire **/dev**. Faites défiler jusqu'au milieu de la sortie. Vous pouvez voir que les fichiers de type bloc commencent par un « b », les fichiers de périphérique de type caractère par un « c » et les fichiers de lien symbolique par un « l » :

```
[analyst@secOps ~]$ ls -l /dev/
<résultat omis>
crw-rw-rw- 1 root tty 5, 2 May 29 18:32 ptmx
drwxr-xr-x 2 root root 0 May 23 06:40 pts
crw-rw-rw- 1 root root 1, 8 May 23 06:41 random
crw-rw-r-- 1 root root 10, 56 May 23 06:41 rfkill
lrwxrwxrwx 1 root root 4 May 23 06:41 rtc -> rtc0
crw-rw---- 1 root audio 253, 0 May 23 06:41 rtc0
brw-rw---- 1 root disk 8, 0 May 23 06:41 sda
brw-rw---- 1 root disk 8, 1 May 23 06:41 sda1
brw-rw---- 1 root disk 8, 16 May 23 06:41 sdb
brw-rw---- 1 root disk 8, 17 May 23 06:41 sdb1
drwxrwxrwt 2 root root 40 May 28 13:47 shm
crw----- 1 root root 10, 231 May 23 06:41 snapshot
drwxr-xr-x 2 root root 80 May 23 06:41 snd
brw-rw----+ 1 root optical 11, 0 May 23 06:41 sr0
lrwxrwxrwx 1 root root 15 May 23 06:40 stderr -> /proc/self/fd/2
lrwxrwxrwx 1 root root 15 May 23 06:40 stdin -> /proc/self/fd/0
lrwxrwxrwx 1 root root 15 May 23 06:40 stdout -> /proc/self/fd/1
crw-rw-rw- 1 root tty 5, 0 May 29 17:36 tty
crw--w---- 1 root tty 4, 0 May 23 06:41 tty0
<résultat omis>
```

- c. Les liens symboliques sous Linux sont l'équivalent des raccourcis dans Windows. Il existe deux types de liens dans Linux : les liens symboliques et les liens matériels. La différence entre ces deux types de liens est qu'un fichier de lien symbolique pointe vers le nom d'un autre fichier, tandis qu'un fichier de lien matériel pointe vers le contenu d'un autre fichier. Créez deux fichiers à l'aide de la commande **echo** :

```
[analyst@secOps ~]$ echo "symbolic" > file1.txt
[analyst@secOps ~]$ cat file1.txt
symbolique
[analyst@secOps ~]$ echo "hard" > file2.txt
```

```
[analyst@secOps ~]$ cat file2.txt
matériel
```

- d. Utilisez la commande **ln -s** pour créer un lien symbolique vers file1.txt et la commande **ln** pour créer un lien matériel vers file2.txt :

```
[analyst@secOps ~]$ ln -s file1.txt file1symbolic
[analyst@secOps ~]$ ln file2.txt file2hard
```

- e. Utilisez la commande **ls -l** et observez le listing du répertoire :

```
[analyst@secOps ~]$ ls -l
total 40
drwxr-xr-x 3 analyst analyst 4096 Aug 16 15:15 cyops_folder2
drwxr-xr-x 2 analyst analyst 4096 Sep 26 2014 Desktop
drwx----- 3 analyst analyst 4096 Jul 14 11:28 Downloads
lrwxrwxrwx 1 analyst analyst 9 Aug 17 16:43 file1symbolic -> file1.txt
-rw-r--r-- 1 analyst analyst 9 Aug 17 16:41 file1.txt
-rw-r--r-- 2 analyst analyst 5 Aug 17 16:42 file2hard
-rw-r--r-- 2 analyst analyst 5 Aug 17 16:42 file2.txt
drwxr-xr-x 8 analyst analyst 4096 Jul 25 16:27 lab.support.files
drwxr-xr-x 3 analyst analyst 4096 Mar 3 18:23 second_drive
-rw-r--r-- 1 analyst analyst 142 Aug 16 15:11 some_text_file.txt
-rw-r--r-- 1 analyst analyst 254 Aug 16 13:38 space.txt
```

Vous remarquez que le fichier **file1symbolic** est un lien symbolique avec un **l** au début de la ligne et un pointeur **->** vers **file1.txt**. Le fichier **file2hard** semble être un fichier ordinaire. En effet, il s'agit d'un fichier ordinaire qui pointe vers le même inode du fichier **file2.txt** sur le disque dur. En d'autres termes, **file2hard** pointe vers les mêmes attributs et emplacements de bloc disque que **file2.txt**.

- f. Modifiez le nom des fichiers d'origine (file1.txt et file2.txt) et regardez ce qu'il se passe au niveau des fichiers liés.

```
[analyst@secOps ~]$ mv file1.txt file1new.txt
[analyst@secOps ~]$ mv file2.txt file2new.txt
```

```
[analyst@secOps ~]$ cat file1symbolic
cat: file1symbolic: no such file or directory
```

```
[analyst@secOps ~]$ cat file2hard
matériel
```

file1symbolic est désormais un lien symbolique rompu, car le nom du fichier vers lequel il pointait (**file1.txt**) a été modifié. Le fichier de lien matériel **file2hard** est quant à lui toujours opérationnel, car il pointe vers l'inode du fichier **file2.txt** et non vers son nom qui est maintenant **file2new.txt**.

D'après vous, qu'arriverait-il au fichier **file2hard** si vous ouvriez un éditeur de texte et modifiez le texte dans **file2new.txt** ?

Remarques générales

La propriété et les autorisations des fichiers sont deux des aspects les plus importants de Linux. Ils sont également souvent à l'origine de problèmes. Un fichier ne détenant pas les bonnes autorisations ou les bons paramètres de propriété ne sera pas accessible aux programmes qui en ont besoin. Dans ce scénario, le programme aura tendance à échouer et des erreurs se produiront.